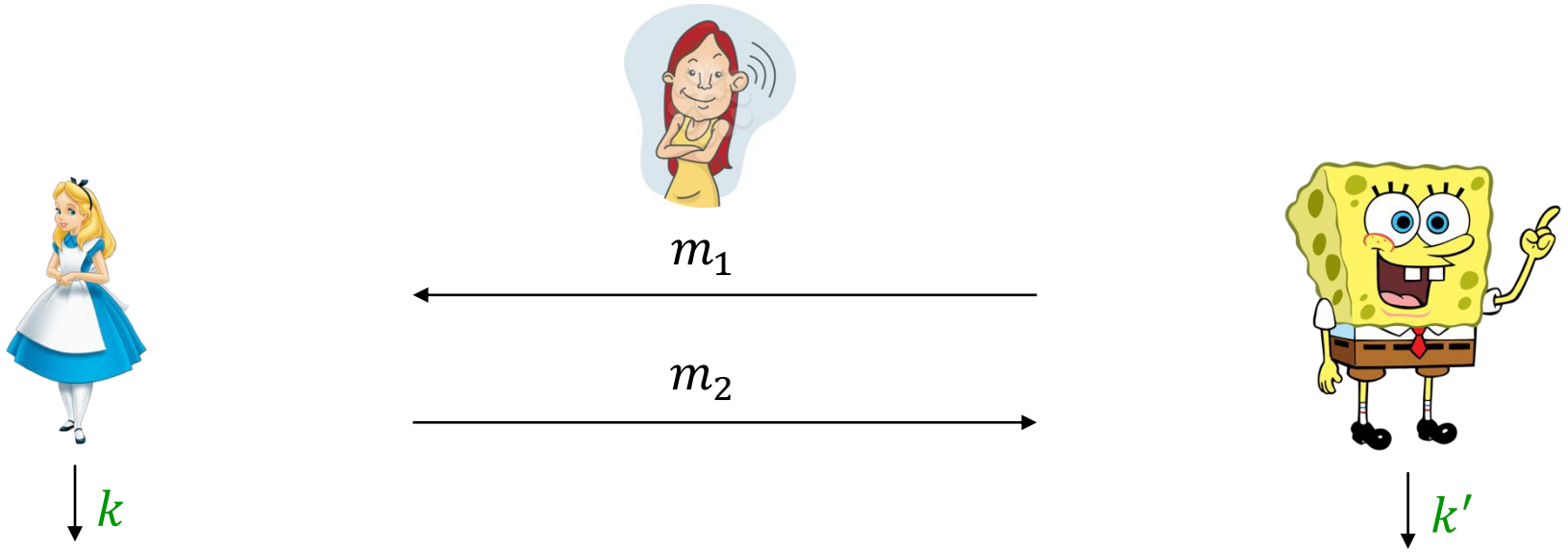
COS 433: Cryptography

Announcements:

- Problem Set 3 due today 10/28
  - Make sure to sign up for check-in 2

- Problem Set 4 to be released today

Key Agreement



- Correctness: if Alice and Bob behave honestly, then $k = k'$
- Weak security: hard for  to find k given (m_1, m_2).
- Strong security: $(m_1, m_2, k) \approx_c (m_1, m_2, u)$ (uniformly random key u)

Diffie-Hellman Key Agreement

p, g known (or sampled by Bob)





$m_1 = g^x$

$m_2 = g^y$



$y \leftarrow \mathbb{Z}_{p-1}$

$k = m_1^y$

$x \leftarrow \mathbb{Z}_{p-1}$

$k' = m_2^x$

- Correctness:

$$k = m_1^y \equiv g^{xy} \equiv m_2^x = k'$$

Computational Diffie-Hellman Assumption over $\mathbb{Z}_p^\times$

Assumption parametrized by a prime p and generator g (usually sampled at random): For all poly-time A,

$$\Pr_{x,y \leftarrow \mathbb{Z}_{p-1}}[A(p, g, g^x, g^y) = g^{xy}] = \mathrm{negl}(\lambda)$$

- Implies the weak security of Diffie-Hellman key agreement
- Open: show this is equivalent to the hardness of discrete log

What about strong security?

Decisional Diffie-Hellman Assumption over $\mathbb{Z}_p^\times$

Assumption parametrized by a prime p and generator g (usually sampled at random):

$$(p, g, g^x, g^y, g^{xy}) \approx_c (p, g, g^x, g^y, g^z)$$

- Implies the strong security of Diffie-Hellman key agreement
- False!
- g^{xy} is a square mod p with probability ¾; g^z is with probability ½

Decisional Diffie-Hellman Assumption over $\mathbb{Z}_p^\times$

Assumption parametrized by a prime p and generator g (usually sampled at random):

$$(p, g, g^x, g^y, g^{xy}) \approx_c (p, g, g^x, g^y, g^z)$$

Now what?

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

Solution: we don't have to work over $\mathbb{Z}_p$!

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

What we need: a "family of effective groups"

- Group $G^{(\lambda)}$ for each λ (or a distribution over groups)
- $\mathrm{poly}(\lambda)$ time group operations
- Easy to sample a group along with a generator g for $G^{(\lambda)}$

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- Need discrete log to be hard
- Need G not to have small subgroups (technically, low-index subgroups)

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- Simple idea: focus on G such that $|G| = q$ is a prime!

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- QR_p where $\frac{p-1}{2}$ is a prime. ("Safe Primes")
- QR_N where $N = pq$ and p, q are safe primes (not prime order).

Decisional Diffie-Hellman Assumption over G

Assumption: parametrized by a cyclic group G and generator g (usually sampled from a distribution)

$$(\langle G \rangle, g, g^x, g^y, g^{xy}) \approx_c (\langle G \rangle, g, g^x, g^y, g^z)$$

In what kinds of groups is DDH plausible?

- QR_p where $\frac{p-1}{2}$ is a prime. ("Safe Primes")
- (Prime order) "elliptic curve groups"

Diffie-Hellman Key Agreement

$G, L = |G|, g$ known or sampled by Bob 



$$m_1 = g^x \quad \longleftarrow$$

$$m_2 = g^y \quad \longrightarrow$$



$y \leftarrow \mathbb{Z}_L$

$\downarrow$

$k = m_1^y$

$x \leftarrow \mathbb{Z}_L$

$\downarrow$

$k' = m_2^x$

Theorem:

- If CDH is true for $(\langle G \rangle, g)$, then protocol is weakly secure
- If DDH is true for $(\langle G \rangle, g)$, then protocol is strongly secure

Diffie-Hellman Key Agreement

$G, L = |G|, g$ known or sampled by Bob 



$y \leftarrow \mathbb{Z}_L$

$k = m_1^y$

$m_1 = g^x$

$m_2 = g^y$



$x \leftarrow \mathbb{Z}_L$

$k' = m_2^x$

Note: you don't even need to know L to run this (details omitted)

ElGamal Encryption

$G, L = |G|, g$ known or sampled by Bob (part of pk)



h

$\mathrm{ct} = g^y, h^y \cdot m$



$m' = \mathrm{ct}_2 \cdot \mathrm{ct}_1^{-x}$

$m \in \{0,1\}^n \subset G$

$y \leftarrow \mathbb{Z}_L$

$x \leftarrow \mathbb{Z}_L$

$\begin{pmatrix} \mathrm{pk} = h = g^x, \\ \mathrm{sk} = x \end{pmatrix}$

PKE based on the DDH assumption; "encryption version" of Diffie-Hellman protocol

- Correctness: $h^y = (g^y)^x$
- Lemma: if the DDH assumption holds, then ElGamal encryption is secure.
- Observe: this scheme is basically running a Diffie-Hellman key agreement to encrypt!

Algorithms for Discrete Log (general groups)

1. Brute force
   - Running time: $|G| \cdot \mathbf{poly}(\lambda) = 2^{\lambda} \cdot \mathbf{poly}(\lambda)$ for $|G| \approx 2^{\lambda}$
2. Baby Step—Giant Step
   - Let $|G| \leq B^2$ (think $B = 2^{\lambda/2}$)
   - Given $(g, h = g^x)$, solve for $0 \leq y, z \leq B$ such that

$$h = g^{y \cdot B + z}$$

   - Enumerate all $g^{y \cdot B}, h \cdot g^{-z}$. Look for a collision!
   - Running time $= B \cdot \mathbf{poly}(\lambda)$. Quadratic speedup ($2^{\lambda/2}$) over brute force.
3. That's all we know!

(Certain) Elliptic curve groups: can set $\lambda = 256$ (pretty small!)

Algorithms for Discrete Log ($\mathbb{Z}_p^\times$ and subgroups)

1. Brute force
   - Running time: $|G| \cdot \mathbf{poly}(\lambda) = 2^\lambda \cdot \mathbf{poly}(\lambda)$ for $|G| \approx 2^\lambda$
2. Baby Step—Giant Step
   - Running time $= B \cdot \mathbf{poly}(\lambda)$. Quadratic speedup ($2^{\lambda/2}$) over brute force.
3. Index Calculus Algorithms
   - Use random-self reduction: $\mathbf{dlog}(g^x) = \mathbf{dlog}(g^{x+r}) - r$
   - Hope that g^{x+r} is "special," develop a tailored algorithm for "special" h
   - Since over $\mathbb{Z}_p^\times$, g^{x+r} is a random integer, one can show it is often special
     - Quadratic Sieve runs in time $2^{O\left(\sqrt{\lambda \cdot \log \lambda}\right)}$
     - Number Field Sieve runs heuristically in time $2^{O\left(\sqrt[3]{\lambda \cdot \log^2(\lambda)}\right)}$

Algorithms for CDH (any group)

1. Solve discrete log over G

2. That's basically all we know!

Algorithms for DDH (any group)

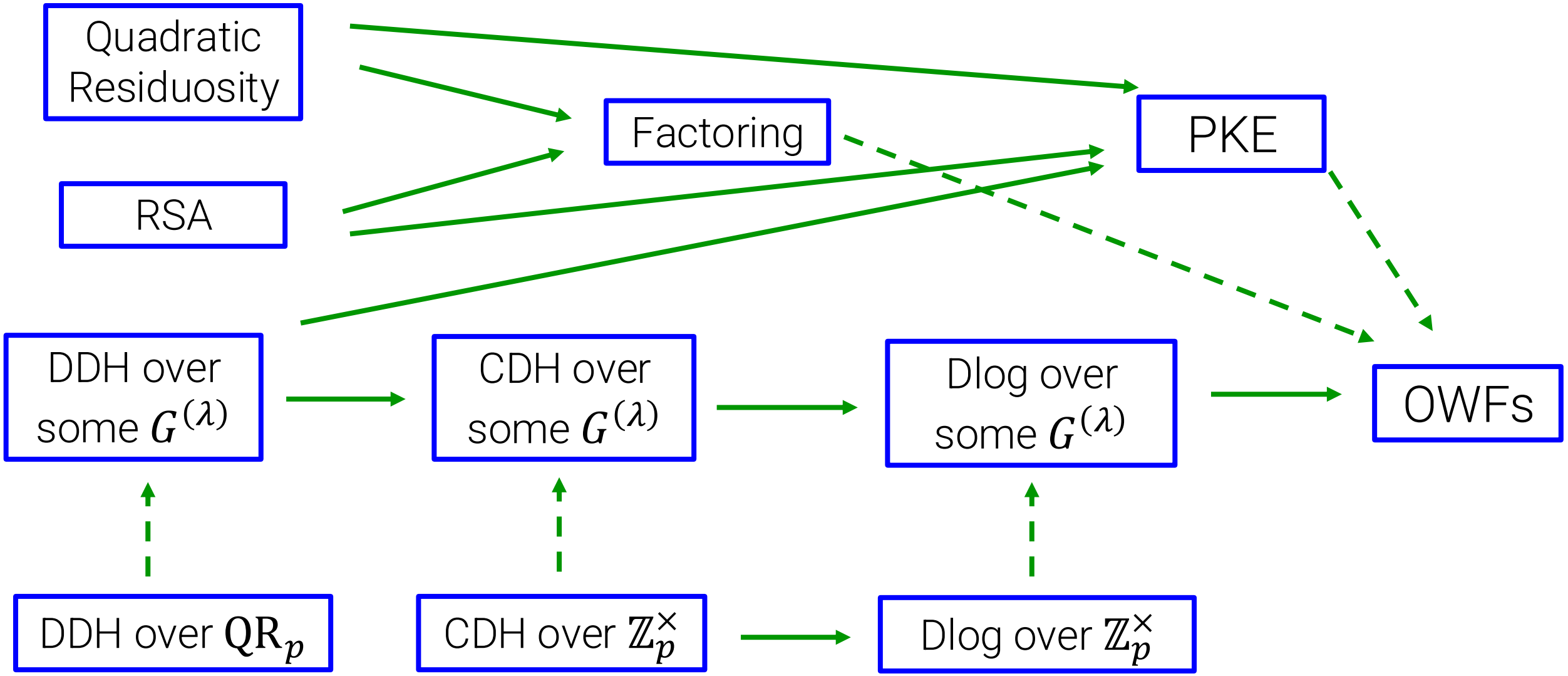
1. Solve discrete log over G

2. Solve discrete log over subgroups (technically, quotients) of G
   - Breaks certain G such as $\mathbb{Z}_p^\times$

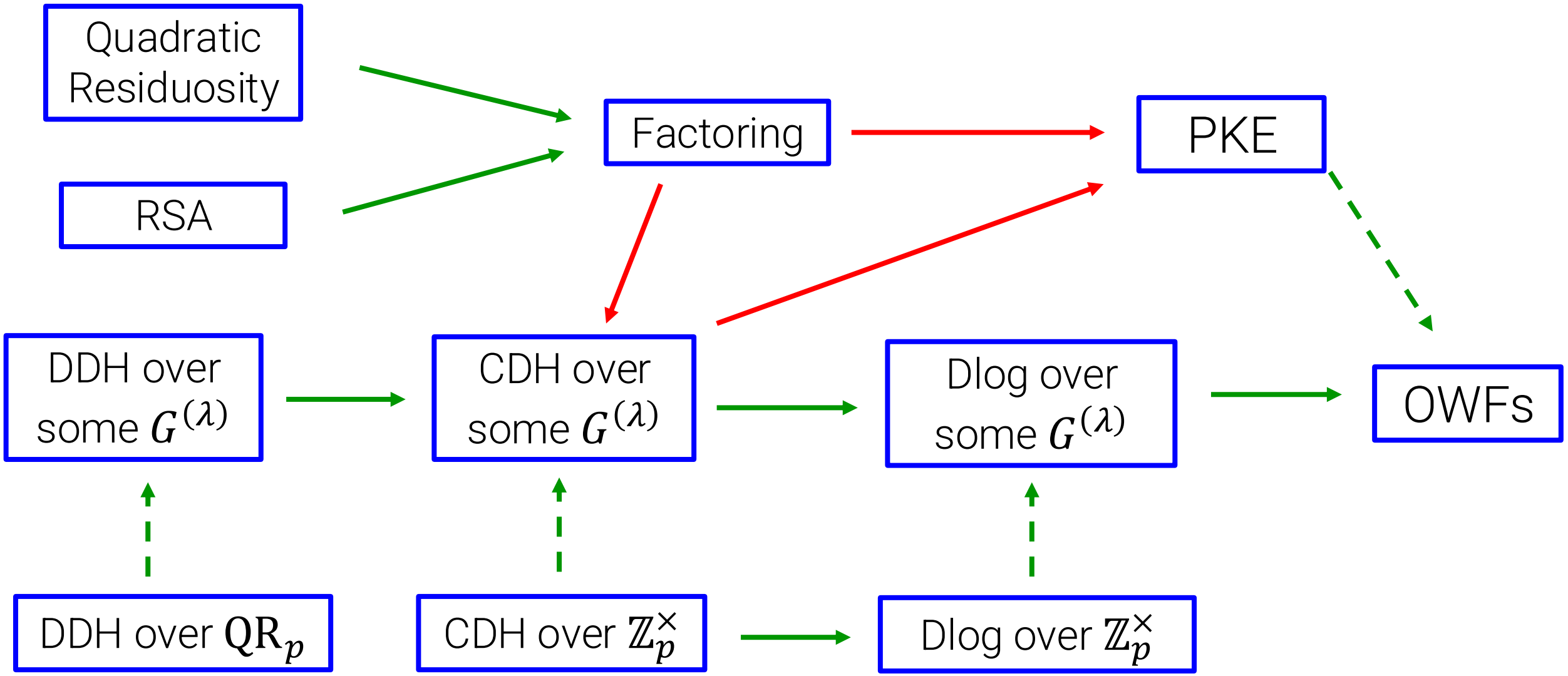
3. It's complicated…
   - DDH broken over certain weird/interesting groups ("self-pairings")

Nevertheless, groups such as QR_p have no known algorithms beyond (1)

Number-Theoretic Hardness Assumptions



Number-Theoretic Hardness Assumptions



End of PKE discussion

Next: digital signatures (authentication)

Public-Key Encryption



$m \in \{0,1\}^n$

pk (broadcast)

$\mathrm{ct} = \mathrm{Enc}(\mathrm{pk}, m; r)$





$m' = \mathrm{Dec}(\mathrm{sk}, \mathrm{ct})$

$(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathrm{Gen}(1^\lambda)$

Problem: man-in-the-middle attacks

If pk broadcast message is tampered with, there is no hope.

Need: a public-key infrastructure. A process indicating that $\mathrm{pk}_{\mathbf{Bob}}$ belongs to some entity named “Bob.”

- We will assume that pk has been recorded to belong to Bob somehow.

Public-Key Encryption



$m \in \{0,1\}^n$

pk (broadcast)

$\text{ct} = \text{Enc}(\text{pk}, m; r)$





$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$

$m' = \text{Dec}(\text{sk}, \text{ct})$

Problem: man-in-the-middle attacks

Main problem: how does Bob know a message was sent by Alice?

Message Authentication Codes (MACs)



k, m



$(m, \sigma = \text{Sign}(k, m))$



k

$\text{Verify}(k, m, \sigma) \in \{0, 1\}$

Definition: A message authentication code (MAC) scheme with message length $n = n(\lambda)$ consists of efficient algorithms $\mathbf{Sign}, \mathbf{Verify}$. It must satisfy the following properties.

- **Correctness:** for all k, m, $\mathbf{Verify}\big(k, m, \mathbf{Sign}(k, m)\big) = 1$ with probability 1.
- **Unforgeability:** for all efficient oracle adversaries $A^{\mathbf{Sign}(k,\cdot)}$, we have:

$$\Pr\left[A^{\mathbf{Sign}(k,\cdot)}(1^\lambda) \to (m^*, \sigma^*) : m^* \text{ was not queried and } \mathbf{Verify}(k, m^*, \sigma^*) = 1\right] = \mathbf{negl}(\lambda)$$

Digital Signatures





k, m

$(m, \sigma = \text{Sign}(k, m))$



k

$\text{Verify}(k, m, \sigma) \in \{0, 1\}$

We want: “public-key MACs”

Digital Signatures



k, m



$(m, \sigma = \text{Sign}(k, m))$



k

$\text{Verify}(k, m, \sigma) \in \{0, 1\}$

Whose key should be public?

Bob's (verification) key

Digital Signatures

$\mathrm{Gen}(1^\lambda) \to (\mathrm{sk}, \mathrm{vk})$



m



vk (broadcast)

$(m, \sigma = \mathrm{Sign}(\mathrm{sk}, m))$



$\mathrm{Verify}(\mathrm{vk}, m, \sigma) \in \{0, 1\}$

Definition: A digital signature scheme with message length $n = n(\lambda)$ consists of efficient algorithms $\mathrm{Gen}, \mathrm{Sign}, \mathrm{Verify}$. It must satisfy the following properties:

- **Correctness:** for all m, $\mathrm{Verify}\big(\mathrm{vk}, m, \mathrm{Sign}(\mathrm{sk}, m)\big) = 1$ with probability 1.
- **Existential Unforgeability:** for all efficient oracle adversaries $A^{\mathrm{Sign}(\mathrm{sk},\cdot)}$, we have:

$$\Pr_{(\mathrm{sk},\mathrm{vk})}\left[A^{\mathrm{Sign}(\mathrm{sk},\cdot)}(\mathrm{vk}) \to (m^*, \sigma^*) : m^* \text{ not queried and } \mathrm{Verify}(\mathrm{vk}, m^*, \sigma^*) = 1\right] = \mathrm{negl}(\lambda)$$

Digital Signatures



$\text{Gen}(1^\lambda) \rightarrow (\text{sk}, \text{vk})$



$m \in \{0,1\}^n$

vk (broadcast)

$(m, \sigma = \text{Sign}(\text{sk}, m))$



$\text{Verify}(\text{vk}, m, \sigma) \in \{0, 1\}$

Notes:

- Message length matters. Not clear how to go from signing one-bit messages to signing longer messages.
- Oracle-based security definition is still required, since the signing key is secret.

$$\Pr_{(\text{sk},\text{vk})}\left[A^{\text{Sign}(\text{sk},\cdot)}(\text{vk}) \rightarrow (m^*, \sigma^*) : m^* \text{ not queried and } \text{Verify}(\text{vk}, m^*, \sigma^*) = 1\right] = \text{negl}(\lambda)$$

“Textbook” RSA Signatures

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$



$m \in \mathbb{Z}_N^\times$



(N, e)

$(m, \sigma = m^d \bmod N)$



$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv m \pmod N$

Notes:

- Restricted to $m \in \mathbb{Z}_N^\times$
- Does not satisfy existential unforgeability. Why?
  - Can multiply any two signatures together to get a new one!
- Secure if the queries and challenge m^* are all random and outside the adversary’s control.

“Textbook” RSA Signatures

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$



$m \in \mathbb{Z}_N^\times$



(N, e)

$(m, \sigma = m^d \bmod N)$



$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv m \pmod N$

Random Message Unforgeability: for all efficient adversaries $A^{\mathrm{Sign}(\mathrm{sk},\cdot)}$ of the form



m^*, vk

“help me”

$m, \mathrm{Sign}(\mathrm{sk}, m)$



$$\Pr_{(\mathrm{sk},\mathrm{vk})}\left[A^{\mathrm{Sign}(\mathrm{sk},\perp)}(m^*, \mathrm{vk}) \to \sigma^* : \mathrm{Verify}(\mathrm{vk}, m^*, \sigma^*) = 1\right] = \mathrm{negl}(\lambda)$$

“Textbook” RSA Signatures

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$





(N, e)

$(m, \sigma = m^d \bmod N)$



$m \in \mathbb{Z}_N^\times$

$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv m \pmod N$

Proof of random message unforgeability: we want



m^*, N, e

”help me”

m, m^d



$$\Pr_{(\mathrm{sk},\mathrm{vk})}\left[A^{\mathrm{Sign}(\mathrm{sk},\perp)}(m^*, N, e) \to (m^*)^d\right] = \mathrm{negl}(\lambda)$$

“Textbook” RSA Signatures

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$



$m \in \mathbb{Z}_N^\times$



(N, e)

$(m, \sigma = m^d \bmod N)$



$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv m \pmod N$

Key Fact:

$$(m, m^d) \equiv (\sigma^e, \sigma)$$

“Textbook” RSA Signatures

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$



$m \in \mathbb{Z}_N^\times$



(N, e)

$(m, \sigma = m^d \bmod N)$



$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv m \pmod N$

Proof of random message unforgeability: equivalently, we want



m^*, N, e

”help me”

σ^e, σ



sk not needed here!

Breaks RSA assumption

$$\Pr_{(\mathrm{sk},\mathrm{vk})}\left[A^{\mathrm{Sign}(\mathrm{sk},\perp)}(m^*, N, e) \to (m^*)^d\right] = \mathrm{negl}(\lambda)$$

Digital Signatures



$\text{Gen}(1^\lambda) \rightarrow (\text{sk}, \text{vk})$



$m \in \{0,1\}^n$

vk

$(m, \sigma = \text{Sign}(\text{sk}, m))$



$\text{Verify}(\text{vk}, m, \sigma) \in \{0, 1\}$

Building full-fledged digital signatures is pretty difficult!

- Practical constructions are usually heuristically secure
- But we do know how to provably build signature schemes (under assumptions)

Digital Signatures



$\text{Gen}(1^\lambda) \rightarrow (\text{sk}, \text{vk})$



$m \in \{0,1\}^n$

vk

$(m, \sigma = \text{Sign}(\text{sk}, m))$



$\text{Verify}(\text{vk}, m, \sigma) \in \{0, 1\}$

Next two classes:

1. One-time signatures (easier to build)
2. RSA signatures (heuristic)
3. DSA/Schnorr signatures (heuristic)